Assignment 1 (Graded)

Project: Building a Full-Stack Node.js Web Server

Project Overview

In this assignment, you will build a functional web server using **Node.js** and the **Express** framework. Your server will serve multi-page HTML content, handle "404, Path Not Found" errors, and process user-submitted data (including file uploads) by saving them to the server's local storage.

Technical Requirements

1. Server Setup & Routing

- **Express Framework:** Initialize a Node.js project and set up an Express server.
- **GET Endpoints (3 Minimum):**
  - `'/'`: Serve a **Home** page.
  - **Two additional paths** of your choice (e.g., `/about`, `/gallery`, or `/contact`).
- **Static Files:** Use middleware to serve static assets (CSS, client-side JS, and Images) from a dedicated directory.
- **Navigation:** Every HTML page must include a navigation menu to allow easy switching between the pages.

- **Error Handling:** Implement a "catch-all" route at the end of your script to redirect any undefined paths to a custom **404 Not Found** page.

2. Data Handling & File Uploads (POST Request)

Create a page containing an HTML form. This form must send a `POST` request to the server with the following:

- **Text Fields:** `u_name`, `u_age`, `u_car`, and `u_job`.
- **File Field:** `u_image` (to upload a profile picture or related image).
- **Server Processing:**
    - Save the text data into a local file (e.g., `submissions.txt`).
    - Save the uploaded image into a folder named `uploads`.
    - *Hint: You will likely need a middleware like `multer` to handle the file upload.*

Recommended Project Structure

To keep your project clean, follow this organizational pattern:

app.js

public

templates

static

uploads

Core Concepts to Master

Before you begin, ensure you understand how the **Request-Response Cycle** works in Express. The server acts as a middleman that listens for a specific "verb" (GET or POST) and a "path," then executes a function to send back a file or save data.

Submission Guidelines

1. Ensure all dependencies are listed in your `package.json`.
2. Test your POST request using the HTML form to ensure files are appearing in the `uploads` folder.
3. You have to submit your node server file (ex app.js), public folder and the package.json

0 % 0 of 1 topics complete

Node js assignment

Assignment